ReachInbox • Outbox Labs Assignment

Full-stack Email Job Scheduler (Production-Grade Service & Dashboard)

Candidate: Tarun Chatrath
Target Role: SDE Intern (Full Stack)
Submission Date: November 2026

1. KEY SUBMISSION DELIVERABLES & LIVE LINKS

🌐 **LIVE DEPLOYED FRONTEND (24/7 CLOUD)**
https://reachinbox-scheduler-frontend-5057.onrender.com

📊 **BULLMQ LIVE QUEUE DASHBOARD**
https://reachinbox-scheduler-backend-5x7n.onrender.com/admin/queues

🐙 **GITHUB REPOSITORY (PRIVATE WITH ACCESS)**
https://github.com/tchatrathbe23-blip/EmailJobScheduler
Collaborators invited: **Mitrajit** & **Yadav036**

⚡ **BACKEND API HEALTH ENDPOINT**
https://reachinbox-scheduler-backend-5x7n.onrender.com/api/health

2. ARCHITECTURAL ARCHITECTURE & TECHNOLOGY STACK

Layer	Technology	Purpose & Implementation
Backend API	TypeScript, Express.js	Modular REST architecture with typed controllers, middleware, and services.
Persistent Queue	BullMQ + Redis	Strictly **NO CRON**. Persistent delayed jobs with concurrency control and backoff retries.
Database & ORM	PostgreSQL (Cloud) / SQLite (Local) via Prisma	Relational models for User, EmailJob, SenderConfig, and RateLimitEvent.
Email Dispatcher	Nodemailer + Ethereal SMTP	Simulated real SMTP email delivery with generated live web preview URLs.
Rate Limiting	Redis Atomic Sliding Counters	Per-sender hourly limits. Automatically reschedules overflow to next hour window.
Slack Alerts	Slack Webhook & OAuth 2.0	Automated Block Kit notifications sent when sender hourly quota is reached.
Search Engine	Elasticsearch 8.x + DB Fallback	Full-text indexing of subject, body, sender, and recipient emails.
Frontend	Next.js 14 (App Router), Tailwind CSS	Modern dark SaaS UI following Figma designs, real-time polling, empty states.
Authentication	Google OAuth 2.0 + Evaluator 1-Click	Real Google Identity verification with JWT sessions, plus fast reviewer sign-in.

3. COMPLIANCE WITH NON-NEGOTIABLE REQUIREMENTS

- ❌ **Absolutely No Cron:** Scheduling relies entirely on BullMQ delayed jobs backed by Redis sorted sets (`zset`). Zero OS-level or node-cron libraries are used.
- 🛡️ **Server Restart Resilience:** When the backend reboots, the `reconcilePendingJobsOnStartup()` engine scans all pending jobs in PostgreSQL/SQLite and synchronizes remaining delay offsets into BullMQ. Future jobs send on time without

duplication.

- ⏱️ **Concurrency & Throttling:** Workers run with configurable concurrency (`WORKER_CONCURRENCY=5`) and enforce a mandatory inter-email delay (`DELAY_BETWEEN_EMAILS_MS`) to simulate provider throttling.
- 🚦 **Zero-Drop Rate Limiter:** Rate limits are calculated using atomic Redis keys (`ratelimit:{sender}:{hourWindow}`). Over-limit jobs are safely rescheduled to the start of the next hour window rather than dropped.
- 🔔 **Live Slack Integration:** Hitting a sender's rate limit triggers a rich Block Kit notification to their connected Slack channel. Also includes a manual "Send Live Alert" test button in the UI.
- 📁 **CSV Lead Parsing:** The Compose modal accepts CSV/text files, extracts valid email addresses via regex, and displays the exact lead count with batch scheduling.

4. FEATURE VERIFICATION MATRIX

Requirement	Status	Implementation Reference
BullMQ Persistent Scheduler (No Cron)	COMPLETED	`backend/src/queues/emailQueue.ts`
Worker Concurrency & Email Delay	COMPLETED	`backend/src/queues/emailWorker.ts`
Hourly Multi-Sender Rate Limiter	COMPLETED	`backend/src/services/rateLimiter.ts`
Server Restart Crash Recovery	COMPLETED	`backend/src/services/schedulerService.ts`
Live Slack Notification on Limit Hit	COMPLETED	`backend/src/services/slackService.ts`
Ethereal SMTP & Web Previews	COMPLETED	`backend/src/services/emailService.ts`
BullMQ Live Web Dashboard	COMPLETED	`backend/src/queues/bullBoard.ts (/admin/queues)`
Elasticsearch Fast Search & Indexing	COMPLETED	`backend/src/services/searchService.ts`
Google OAuth & Evaluator Login	COMPLETED	`backend/src/controllers/authController.ts`
Next.js 14 Dashboard + Compose CSV	COMPLETED	`frontend/src/app/dashboard/page.tsx`
Docker Compose & Render Cloud Deploy	COMPLETED	`docker-compose.yml & render.yaml`